

Getting Started Guide

Overview

To reproduce results in csgc paper, one needs two servers and an [Daisy+ OpenSSD](#). Assume that we have server A and B and OpenSSD is installed on A and connected to B for programming and debugging.

The benchmark routines are as follow:

- Firstly, on server B we program the OpenSSD firmware and boot OpenSSD.
- Secondly, we boot server A and then run all the benchmarks to evaluate csgc.

Folder structures:

csgc-artifacts

- **host** (host-side source codes and benchmark tools)
 - **benchmarks** (benchmarks, scripts and tools for reproducing results)
 - **src** (source codes for csgc, iplfs and a customized nvme-cli)
 - **f2fs-tools-csgc** (used by csgc to format the storage)
 - **f2fs-tools-iplfs** (used by iplfs to format the storage)
 - **linux-csgc** (kernel source codes for CSGC, can be configured to run in csgc mode or vanilla-f2fs mode)
 - **linux-iplfs** (kernel source codes for IPLFS)
 - **nvme-cli** (a customized NVMe CLI with commands to configure openssd and get statistics)
- **openssd** (device-side source codes)
 - **scripts** (scripts used to move codes from **src** into vitis workspace)
 - **src** (the openssd controller logic source code, including csgc execution and sFTL)
 - **vitis_export_archive_csgc.ide.zip** (the vitis workspaces including controller logic and hardware design)

If you encounter any problems during the evaluation, please contact grey-hibari@sjtu.edu.cn.

OpenSSD Setup

****These steps happen on server B****

Step 1: Install Vivado and Vitis

To use OpenSSD, you need to install Vivado 2023.1, which is available [here](#).

Follow the guides in the installer, and before installation starts, be sure to select all the boxes under "Vitis Unified Software Platform" and "Devices for Custom Platforms".

Step 2: Import Vitis Workspace

Open Vivado, and then open Vitis through "Tools -> Launch Vitis IDE".

In Vitis, click "File -> Import...", and select "Vitis project exported zip file", then import `csgc-artifacts/openssd/vitis_export_archive_csgc.ide.zip`.

Step3: Compile and Run

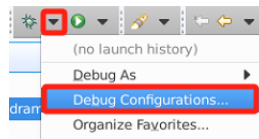
Select the `nvme_system` project and build it, which should take about 1 minute.

Then add a "Vitis Serial Terminal" view by clicking "Window->show view..." and searching for "Vitis Serial Terminal".

In the added view, click the "+" button to add the USB port where OpenSSD is connected to, so that the outputs can be displayed in the VST.

Before proceeding further, **make sure server A is shut down**.

Finally, click the "Debug Configuration" button shown as below, and double click the "System Project Debug" to create a debug configuration. Then click the "Debug" button.



You should see all 4 cores stop at the entry of main function. Let them continue, and some initialization logs would show in the VST. When you see "Turn on the host PC", you can turn on server A.

After A is on, run `lsblk` and you should see an NVMe device with 59.8GB size in the output.

Host Setup

****These steps happen on server A****

Step 1: Build Linux Kernels

Build the CSGC kernel in `host/src/linux-csgc` and the IPLFS kernel in `host/src/linux-iplfs`. You should switch to the right kernel according to the benchmark configs.

Please use the following commands to build and install the kernel:

```
cp /boot/config-$(uname -r) .config
make olddefconfig
make -j$(nproc) LOCALVERSION=-csgc V=1 2>&1 | tee kernel_build_log # use LOCALVERSION=-iplfs for IPLFS
sudo make -j$(nproc) modules_install
sudo make install
```

Step 2: Prepare Benchmark Environments

Firstly install cgroup-tools, which will be used to constrain memory usage:

```
sudo apt install cgroup-tools=2.0-2
```

Secondly, build customized nvme-cli in `host/src/nvme-cli`:

```
cd host/src/nvme-cli  
make
```

Thirdly, build and install filebench and fio:

```
# Filebench (Check for dependencies in host/benchmarks/filebench/README)  
cd host/benchmarks/filebench  
libtoolize  
aclocal  
autoheader  
automake --add-missing  
autoconf  
  
./configure  
make  
sudo make install  
  
# fio  
cd host/benchmarks/fio  
./configure  
make  
sudo make install
```

Finally, prepare environments for ycsb

```
# install mysql, python2 and java  
sudo apt-get install mysql-server=8.0.42-0ubuntu0.22.04.1  
sudo apt install python2  
sudo apt install openjdk-11-jre-headless
```

Step 3: Load Data for YCSB Benchmark

We need to load ycsb data in advance, since loading records during the evaluation can significantly increase evaluation time. Run the load scripts as below:

```
cd host/benchmarks/scripts
# make sure your mysql data path is /var/lib/mysql (which is the default)
# make sure you dont have any important data in mysql, since the following script will
delete all data in the mysql
sudo ./load_ycsb.sh
```

The loading process will take about one and a half minute.

After loading completes, change the data path of mysql to `<MOUNT_POINT>/mysql`, `<MOUNT_POINT>` is the path where you are going to mount OpenSSD. We recommend using the same path as we do, i.e.

`<MOUNT_POINT> = /mnt/openssd_f2fs`. And the mysql data path would be `/mnt/openssd_f2fs/mysql`.

When running ycsb benchmark, the db data will be copied from `/var/lib/mysql` to `/mnt/openssd_f2fs/mysql`.

Step-by-Step Instructions

We use the following platform for evaluation:

Component	Specification
CPU	Two Intel Xeon Gold 6240 CPUs @ 2.60GHz
Memory	384GB
Storage	Daisyplus OpenSSD with 64GB DRAM as storage backend
Linux Distribution	Ubuntu 22.04.5 LTS

To reproduce all the results in the paper, we have 3 system to evaluate: **vanilla F2FS**, **IPLFS**, and **CSGC**(our system). For each system, we would run the script `host/benchmarks/scripts/test.sh` with all 10 configs in the folder `host/benchmarks/scripts/configs`.

Before running the scripts, you should check the mount point of your OpenSSD, if it is not `/mnt/openssd_f2fs`, you need to modify the following line (`host/benchmarks/scripts/common.sh`, line 17)

```
MNTPOINT=/mnt/openssd_f2fs # modify this to your mount point
```

Full Evaluation

A full evaluation may take >10 hours. If you only want to reproduce partial results, see the **Partial Evaluation** section

Step 1: Switch to CSGC kernel

Firstly, please switch to CSGC kernel.

After the kernel is installed, you may find its boot option in `/boot/grub/grub.cfg`, there should be lines like these:

```
submenu 'Advanced options for Ubuntu' $menuentry_id_option 'gnulinux-advanced-973e43f7-8b45-444f-b47c-b12722dbb34a' {  
  ...  
  menuentry 'Ubuntu, with Linux 6.1.54-csgc' --class ubuntu --class gnu-linux --class gnu  
  --class os $menuentry_id_option 'gnulinux-6.1.54-csgc-advanced-973e43f7-8b45-444f-b47c-b12722dbb34a'  
  ...  
}
```

Then modify `/etc/default/grub`, set the `GRUB_DEFAULT` variable to the newly installed CSGC kernel like this:

```
GRUB_DEFAULT="gnulinux-advanced-973e43f7-8b45-444f-b47c-b12722dbb34a>gnulinux-6.1.54-csgc-advanced-973e43f7-8b45-444f-b47c-b12722dbb34a"
```

Then run `sudo update-grub` and reboot.

Now, run `uname -r` and the output should be `6.1.54-csgc`

Step 2: Evaluate CSGC

Use the following commands to run all benchmarks for CSGC. **We strongly recommend you to reboot the server before running each command**, which will greatly reduces the chance of failure.

```
cd host/benchmarks/scripts/  
  
sudo ./test.sh cs configs/config01_filebench_a.sh  
sudo ./test.sh cs configs/config02_filebench_b.sh  
sudo ./test.sh cs configs/config03_filebench_c.sh  
sudo ./test.sh cs configs/config04_ycsb_a.sh  
sudo ./test.sh cs configs/config05_ycsb_f.sh  
sudo ./test.sh cs configs/config06_fio_rand.sh  
sudo ./test.sh cs configs/config07_fio_skew.sh  
sudo ./test.sh cs configs/config08_fio_storage_util.sh  
sudo ./test.sh cs configs/config09_fio_secsz.sh  
sudo ./test.sh cs configs/config10_fio_skewness.sh
```

Step 3: Evaluate vanilla F2FS

You don't need to switch kernel to evaluate vanilla F2FS, since the csgc kernel can be configured to run in vanilla F2FS mode. Run the following commands (the vanilla kernel is stable, and you don't need to reboot between each command) :

```
sudo ./test.sh ori configs/config01_filebench_a.sh
sudo ./test.sh ori configs/config02_filebench_b.sh
sudo ./test.sh ori configs/config03_filebench_c.sh
sudo ./test.sh ori configs/config04_ycsb_a.sh
sudo ./test.sh ori configs/config05_ycsb_f.sh
sudo ./test.sh ori configs/config06_fio_rand.sh
sudo ./test.sh ori configs/config07_fio_skew.sh
sudo ./test.sh ori configs/config08_fio_storage_util.sh
sudo ./test.sh ori configs/config09_fio_secsz.sh
sudo ./test.sh ori configs/config10_fio_skewness.sh
```

Step 4: Switch to IPLFS kernel

Similar as step 1, find the boot option for iplfs kernle, modify the grub setting and reboot.

The `uname -r` output should be `5.11.0-iplfs`

Step 5: Evaluate IPLFS

Use the following commands to run all benchmarks for IPLFS. **We strongly recommend you to reboot the server before running each command**, which will greatly reduces the chance of failure.

```
sudo ./test.sh iplfs configs/config01_filebench_a.sh
sudo ./test.sh iplfs configs/config02_filebench_b.sh
sudo ./test.sh iplfs configs/config03_filebench_c.sh
sudo ./test.sh iplfs configs/config04_ycsb_a.sh
sudo ./test.sh iplfs configs/config05_ycsb_f.sh
sudo ./test.sh iplfs configs/config06_fio_rand.sh
sudo ./test.sh iplfs configs/config07_fio_skew.sh
sudo ./test.sh iplfs configs/config08_fio_storage_util.sh
sudo ./test.sh iplfs configs/config09_fio_secsz.sh
sudo ./test.sh iplfs configs/config10_fio_skewness.sh
```

Step 6: Generate Plots

Run the python scripts in `host/benchmarks/scripts/plot`, the generated figures will be placed in `host/benchmarks/scripts/figs` .

```
# matplotlib and numpy is needed for the python scripts to work
python3 plots/1perf_overall.py      # figure 4
python3 plots/2timeline_lat.py      # figure 5
python3 plots/3fio_storage_util.py  # figure 6
python3 plots/4fio_secsz.py         # figure 7
python3 plots/5fio_skewness.py      # figure 8
```

Partial Evaluation

Table below shows the minimal set of configs needed to reproduce each figure in the paper and the time it takes for the evaluation. Note that you need to run the set of configs for all three systems, as shown in the **Full Evaluation** section. To see how to run a set of configs for one system, check **step 2, 3 and 5** in the **Full Evaluation** section.

Config Set	Results in the paper	Full Execution Time for 3 systems
Config 2 ~ 7	Figure 4	~ 3 hours
Config 1, Config 4	Figure 5	~ 1 hour
Config 8	Figure 6	~ 2.5 hours
Config 9	Figure 7	~ 2.5 hours
Config 10	Figure 8	~ 3 hours

Unfortunately, it is hard to avoid the long evaluation time since it is necessary to fill the storage to almost full before the evaluation.

If you want to reduce the evaluation time, you can set the `light_evaluation` variable to 1 (In `host/benchmarks/scripts/test.sh`, line 6). This will reduce evaluation time (not by much though :)) for filebench and ycsb benchmark by reducing runtime from 300s to 60s for filebench and reducing number of operations from 2 million to 1 million for ycsb.

With `light_evaluation=1`, CSGC may show less performance gain due to less GC counts.

Output Explanation

Each config in `host/benchmarks/scripts/configs` contains a (group of) workload(s). For each config, the `test.sh` script will generate a folder named with the time of the evaluation. Under this folder is log folders of each workload inside the config. The outputs folder for CSGC, IPLFS, and vanilla F2FS are `host/benchmarks/scripts/outputs-cs`, `host/benchmarks/scripts/outputs-iplfs` and `host/benchmarks/scripts/outputs-cs`, respectively.

For example, running config8, which contains 5 workloads with different storage utility, with CSGC will generate outputs like this:

```

outputs-cs/20250512_152555/
├─ fio_randwrite_s8_0.6_random
├─ fio_randwrite_s8_0.7_random
├─ fio_randwrite_s8_0.8_random
├─ fio_randwrite_s8_0.95_random
└─ fio_randwrite_s8_0.9_random

# the output folder is named in the following format:
# <bm_type>_<bm_name>_s<secsz>_<storage_util>[_<random_pattern>]
# <bm_type> = filebench | ycsb | fio
# <bm_name> = ... (check this folder: host/benchmarks/myworkloads)
# <secsz> : F2FS section size

```

And under each output folder is the logs generated by that workload:

```

outputs-cs/20250512_152555/fio_randwrite_s8_0.8_random/
├─ dmesg.log # kernel output, for debug use
├─ dmesg.old
├─ fio.log # output of the benchmark, performance info comes from here
└─ stat.log # statistics collected from kernel and OpenSSD

```

In the benchmark output log (<bm_type>.log), you can see detailed performance information including throughput and latency. In the statistics log (stat.log), you can see the statistics collected from kernel (the latency of GC) and OpenSSD (the I/O statistics and write amplification).

With `light_evaluation=1`, we have run config 1 ~ 7 for all 3 systems, and config 8 ~ 10 for CSGC. You can check the outputs in the `outputs-{cs,iplfs,origc}` folders. With these data, you can run `plots/lperf_overall.py` and `plots/2timeline_lat.py` to generate figures.